

NATIONAL ECONOMICS UNIVERSITY – FACULTY OF DATA SCIENCE
AND AI

DSAI1002: Data Structures and Algorithms with Python

LECTURE NOTES: PART 04
Sorting, Selection, and Searching Algorithms

Author / Instructor: Dr. Minh D. Vu | Term: Fall 2026

Comprehensive Academic Monograph on Sorting Models, Comparison vs. Non-Comparison Algorithms, Order Statistics, Binary/Ternary Searching, and Practical Python Integration

Executive Roadmap of Part 04

This monograph synthesizes the theoretical concepts, formal algorithms, step-by-step traces, and complexity analyses presented across **Lecture 4**:

- Foundations of Sorting:** Motivation, mathematical specification, core taxonomy (stability, in-place, adaptiveness, online vs. offline, comparison vs. integer-key).
- Elementary Comparison Sorts:** Insertion Sort (card-hand model, loop invariant, $\Theta(n)$ best case, $\Theta(n^2)$ worst case) and Bubble Sort (adjacent swaps, early termination flag, $\Theta(n^2)$ worst case).
- Divide-and-Conquer Sorting:** Merge Sort (two-way merge buffer, $\Theta(n \log n)$ guaranteed, stability, $O(n)$ extra space) and Quick Sort (Lomuto partition, pivot selection strategies, worst-case avoidance, expected $O(n \log n)$ in-place).
- Non-Comparison Linear Sorts:** The $\Omega(n \log n)$ decision tree lower bound, Counting Sort ($\Theta(n + k)$ stable index mapping), and Radix Sort (multi-pass LSD stable digit distribution, $\Theta(d(n + b))$).
- Python Sorting Ecosystem:** Timsort mechanics, 'sorted()' vs. 'list.sort()', multi-key ranking, lambda keys, and common sorting bugs.
- The Selection Problem:** Order statistics, extreme values, Quickselect (expected $\Theta(n)$ average rank retrieval), the Median of Medians algorithm (deterministic $O(n)$ worst case), and Top- k strategies.
- Searching Algorithms & Optimization:** Linear search, Binary Search (invariants, overflow avoidance, first/last occurrence variants, 'bisect'), Ternary Search on unimodal functions and discrete arrays, and data structure selection tradeoffs.

Contents

1	The Sorting Problem and Core Algorithmic Properties	4
1.1	Why Study Sorting Algorithms?	4
1.2	Formal Mathematical Problem Specification	4
1.3	Essential Taxonomy and Properties of Sorting Algorithms	4
2	Elementary Comparison Sorts: Insertion Sort and Bubble Sort	6
2.1	Insertion Sort	6
2.1.1	Algorithm and Pseudocode	6
2.1.2	Step-by-Step Execution Trace	6
2.1.3	Formal Complexity and Properties	6

2.2	Bubble Sort	7
2.2.1	Step-by-Step Execution Trace with Early Exit	7
2.2.2	Bubble Sort versus Insertion Sort	7
2.3	Worked Examples and Inversion Analysis for Elementary Sorts	8
3	Efficient Divide-and-Conquer Sorts: Merge Sort and Quick Sort	9
3.1	Merge Sort	9
3.1.1	Formal Merge and Merge Sort Pseudocode	9
3.1.2	Mathematical Complexity Analysis	9
3.2	Quick Sort	10
3.2.1	Lomuto Partition Scheme and Quicksort Pseudocode	10
3.2.2	Trace of Lomuto Partitioning	10
3.2.3	The Pivotal Role of Pivot Selection	10
3.3	Worked Examples on Divide-and-Conquer Sorting	11
4	Non-Comparison Linear-Time Sorting: Counting Sort & Radix Sort	12
4.1	The $\Omega(n \log n)$ Lower Bound for Comparison Sorts	12
4.2	Counting Sort	12
4.2.1	Why Counting Sort Must Scan Backwards for Stability	12
4.2.2	Complexity Analysis and Constraints	13
4.3	Radix Sort	13
4.4	Worked Examples: Counting Sort and Radix Sort Traces	14
5	Python Sorting in Practice	15
5.1	Python's Built-in Sorting: 'sorted()' vs. 'list.sort()'	15
5.1.1	Multi-Attribute Sorting with Keys	15
5.2	Practical Python Sorting Case Studies	15
5.3	Common Sorting Mistakes and Anti-Patterns	16
6	The Selection Problem and Order Statistics	17
6.1	Order Statistics and Ranks	17
6.2	Finding Extremes in a Single Scan: $\Theta(n)$	17
6.3	Strategies for General Rank Selection	17
6.4	Quickselect (Hoare's Selection Algorithm)	17
6.4.1	Trace for 5th-Smallest Value ($k = 5 \implies$ target index 4)	18
6.4.2	Quickselect Complexity Analysis	18
6.5	The Median of Medians Algorithm (BFPRT)	18
6.6	Worked Examples in Selection and Top- k Extraction	18
7	Searching Algorithms: Linear, Binary, and Ternary Search	20
7.1	Linear Search	20
7.2	Binary Search	20
7.2.1	Trace for Target 6 on [2, 3, 4, 5, 6, 7]	20
7.2.2	Correctness Invariant and Complexity	20
7.3	Binary Search Variants: First and Last Occurrences	20
7.4	Python's 'bisect' Library	21
7.5	Ternary Search on Unimodal Functions	21
7.6	Comprehensive Data Structure Selection Matrix for Lookups	21
7.7	Worked Examples in Searching Algorithms	21
8	Synthesis Problems and Worked Solutions	23
8.1	Problem 1: Transaction Record Multi-Key Sorting	23
8.2	Problem 2: Choosing Top- k Strategy	23
8.3	Problem 3: Two-Sum on a Sorted Array	23

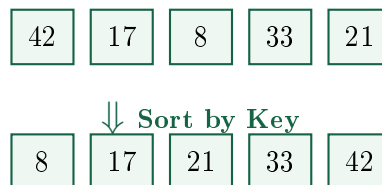
8.4 Problem 4: Searching in a Rotated Sorted Array 24

1. The Sorting Problem and Core Algorithmic Properties

1.1 Why Study Sorting Algorithms?

In computational systems, raw data rarely arrives in an organized manner. Sorting rearranges items into a predictable sequence. While sorting is rarely an end in itself, it functions as a critical foundational utility:

- **Search Acceleration:** In an unsorted array of size n , finding an item requires linear search ($O(n)$). Once sorted, searching requires only $O(\log n)$ via binary search (reducing 1,000,000 checks to just 20 comparisons).
- **Duplicate Detection:** In unsorted data, detecting duplicates takes $O(n^2)$ naive checks or extra hash tables. In sorted data, identical elements are contiguous and can be found in a single $O(n)$ pass.
- **Database and Index Operations:** Relational joins (Merge Join), grouping, and range queries rely fundamentally on sorted inputs.
- **Ubiquitous Applications:** E-commerce product listings (by price or rating), financial transactions (chronological audit trails), student GPA rankings, and airline flight departures.



1.2 Formal Mathematical Problem Specification

Definition 1.1 (The Sorting Problem)

Given an input sequence of n records $A = \langle a_0, a_1, \dots, a_{n-1} \rangle$ and a key extraction function $k : A \rightarrow \mathcal{K}$ associated with a total ordering relation $\leq$ on $\mathcal{K}$, compute a permutation $A' = \langle a'_0, a'_1, \dots, a'_{n-1} \rangle$ of A such that:

$$k(a'_0) \leq k(a'_1) \leq k(a'_2) \leq \dots \leq k(a'_{n-1})$$

Correctness Invariant: (1) The output sequence must be monotonically non-decreasing with respect to key k , and (2) A' must be a strict permutation of A (no items created, lost, or duplicated).

1.3 Essential Taxonomy and Properties of Sorting Algorithms

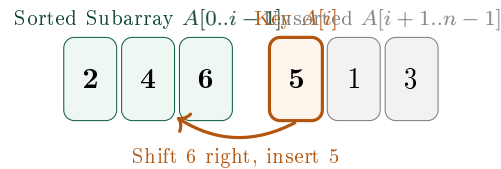
To evaluate and select an appropriate sorting algorithm, computer scientists examine five fundamental properties:

Property	Formal Algorithmic Definition
Stability	A sort is stable if records with equal keys appear in the output in the exact same relative order as in the input. If $k(A[i]) = k(A[j])$ and $i < j$, then $A[i]$ must appear before $A[j]$ in the sorted result. Essential for multi-attribute sorting (e.g., sort by name, then stably sort by GPA).
In-Place (In Situ)	An algorithm is in-place if it sorts using only $O(1)$ auxiliary storage beyond the input array (or $O(\log n)$ memory on the recursion call stack). It rearranges elements within the original array without allocating a full duplicate buffer.
Adaptiveness	An algorithm is adaptive if its running time improves when the input already possesses partial or complete presortedness (e.g., Insertion Sort runs in $\Theta(n)$ time when inversions $I = O(n)$).
Comparison Sort	The algorithm accesses keys exclusively through pairwise comparisons ($k(a) \leq k(b)$). It treats keys as opaque values and is subject to the information-theoretic lower bound $\Omega(n \log n)$.
Integer-Key Sort	The algorithm exploits the internal bit/digit structure of keys (e.g., counting, digit distribution) to index directly into auxiliary tables, breaking the comparison bound to achieve linear $\Theta(n)$ performance.
Online Capability	The algorithm can process and maintain sorted order incrementally as items arrive one by one from a live data stream, without needing the entire dataset upfront.

2. Elementary Comparison Sorts: Insertion Sort and Bubble Sort

2.1 Insertion Sort

Conceptual Intuition: Insertion Sort mirrors the way human players organize a hand of playing cards. Starting with an empty left hand and cards on the table, we pick one card at a time and insert it into its correct relative position among the already sorted cards in our hand.



2.1.1 Algorithm and Pseudocode

At step i , element ‘key = $A[i]$ ’ is compared against elements in the sorted prefix $A[0..i-1]$ from right to left. Elements strictly greater than ‘key’ are shifted one position to the right to create an opening, and ‘key’ is inserted.

```

1 FUNCTION INSERTION_SORT(A)
2 INPUT: array A of length n
3 OUTPUT: array A sorted in non-decreasing order
4   FOR i = 1 TO n - 1 DO
5     key = A[i]
6     j = i - 1
7     WHILE j >= 0 AND A[j] > key DO
8       A[j + 1] = A[j]      // Shift element to the right
9       j = j - 1
10    A[j + 1] = key         // Place key into opening
11  RETURN A

```

2.1.2 Step-by-Step Execution Trace

Consider input array $A = [7, 3, 5, 2, 6, 4]$ ($n = 6$):

Pass (i)	Key ($A[i]$)	Array State After Pass	Internal Shift Operations
Start	–	[7, 3, 5, 2, 6, 4]	Prefix $A[0..0] = [7]$ is trivially sorted
$i = 1$	3	[3, 7, 5, 2, 6, 4]	$7 > 3 \implies 7$ shifts right; 3 inserted at index 0
$i = 2$	5	[3, 5, 7, 2, 6, 4]	$7 > 5 \implies 7$ shifts right; 5 inserted at index 1
$i = 3$	2	[2, 3, 5, 7, 6, 4]	$7, 5, 3 > 2 \implies$ all shift right; 2 inserted at index 0
$i = 4$	6	[2, 3, 5, 6, 7, 4]	$7 > 6 \implies 7$ shifts right; 6 inserted at index 3
$i = 5$	4	[2, 3, 4, 5, 6, 7]	$7, 6, 5 > 4 \implies$ shift right; 4 inserted at index 2

2.1.3 Formal Complexity and Properties

- **Loop Invariant:** At the start of each iteration i , the subarray $A[0..i-1]$ consists of the original elements in sorted order.
- **Best-Case Time Complexity:** $\Theta(n)$. When the array is already sorted, $A[j] > \text{key}$ evaluates to ‘False’ on the very first comparison for every i . The inner loop executes in $O(1)$ time, yielding

$$T(n) = \sum_{i=1}^{n-1} 1 = n - 1 = \Theta(n).$$

- **Worst-Case Time Complexity:** $\Theta(n^2)$. When the array is reverse sorted, every element must shift past all i predecessors:

$$T_{\text{worst}}(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

- **Average-Case Time Complexity:** $\Theta(n^2)$. On average, an element shifts through half of the sorted prefix ($i/2$ comparisons), yielding $\approx n^2/4$ operations.
- **Auxiliary Space:** $\Theta(1)$ (In-place).
- **Stability: Stable.** The condition $A[j] > \text{key}$ uses a strict inequality; when $A[j] == \text{key}$, the while loop terminates, preserving original order.

2.2 Bubble Sort

Conceptual Intuition: Adjacent pairs of elements are compared sequentially. If out of order, they are swapped. Through repeated sweeps, large elements “bubble up” toward the right end of the array, fixing the largest element in place at the end of each pass.

```

1 FUNCTION BUBBLE_SORT(A)
2 INPUT: array A of length n
3 OUTPUT: array A sorted in non-decreasing order
4   FOR end = LENGTH(A) - 1 DOWNTO 1 DO
5     swapped = FALSE
6     FOR i = 0 TO end - 1 DO
7       IF A[i] > A[i + 1] THEN
8         SWAP A[i], A[i + 1]
9         swapped = TRUE
10    IF swapped == FALSE THEN
11      RETURN A // Early exit if array is already sorted
12  RETURN A

```

2.2.1 Step-by-Step Execution Trace with Early Exit

Trace on array $A = [7, 3, 5, 2, 6, 4]$:

Pass	Fixed Suffix	Array State After Pass	Swaps Performed
Start	none	[7, 3, 5, 2, 6, 4]	Initial state
1	[7]	[3, 5, 2, 6, 4, 7]	(7, 3), (7, 5), (7, 2), (7, 6), (7, 4) all swap
2	[6, 7]	[3, 2, 5, 4, 6 , 7]	(5, 2), (6, 4) swap
3	[5, 6, 7]	[2, 3, 4, 5 , 6 , 7]	(3, 2), (5, 4) swap
4	[4, 5, 6, 7]	[2, 3, 4 , 5 , 6 , 7]	No swaps performed! Early exit triggered.

2.2.2 Bubble Sort versus Insertion Sort

Although both have worst-case $O(n^2)$ and best-case $O(n)$ with early termination, **Insertion Sort is vastly superior in practice**. Bubble Sort requires three memory writes for every adjacent inversion swap (‘temp = a; a = b; b = temp’), whereas Insertion Sort shifts elements with a single write (‘A[j+1] = A[j]’) and inserts the key only once per outer iteration.

2.3 Worked Examples and Inversion Analysis for Elementary Sorts

Example 2.1 (Predicting Work on Partially Sorted Data)

Consider running Insertion Sort and Bubble Sort on three arrays of size $n = 6$:

1. **Case A (Already Sorted):** $A_1 = [2, 3, 4, 5, 6, 7]$.
 - *Insertion Sort:* Each $A[i]$ immediately satisfies $A[i - 1] \leq A[i]$. Only $n - 1 = 5$ comparisons and 0 shifts occur. Total time: $\Theta(n)$.
 - *Bubble Sort (Optimized):* Pass 1 compares all 5 adjacent pairs. No swaps occur; ‘swapped’ remains ‘False’. The algorithm terminates after 5 comparisons and 0 swaps. Total time: $\Theta(n)$.
2. **Case B (Single Misplaced Item at Front):** $A_2 = [7, 2, 3, 4, 5, 6]$.
 - *Insertion Sort:* For $i = 1$, element 2 shifts past 7. For all subsequent $i \geq 2$, elements are already in order. Total comparisons: $1 + 1 + 1 + 1 + 1 = 5$. Total shifts: 1. Total time: $O(n)$.
 - *Bubble Sort:* Value 7 must bubble across the entire array in Pass 1 (5 swaps). Pass 2 checks all remaining pairs (no swaps) and terminates. Total comparisons: $5 + 4 = 9$, total swaps: 5.
3. **Case C (Strictly Reverse-Sorted):** $A_3 = [7, 6, 5, 4, 3, 2]$.
 - *Insertion Sort:* Maximum possible shifts. Total comparisons and shifts: $\sum_{i=1}^5 i = 15$. Total time: $\Theta(n^2)$.
 - *Bubble Sort:* Maximum possible passes ($n - 1 = 5$). Total comparisons: $5 + 4 + 3 + 2 + 1 = 15$, total swaps: 15.

Theorem 2.1 (Inversion Count Connection)

An **inversion** in an array A is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. If an array contains I inversions:

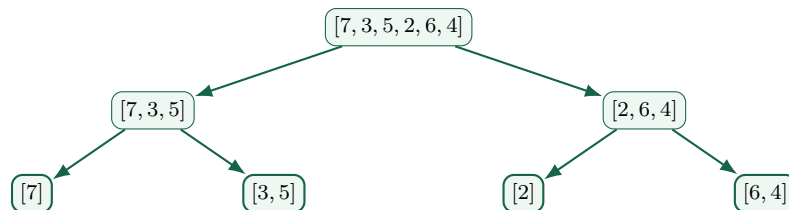
- Every adjacent swap in Bubble Sort or shift in Insertion Sort eliminates exactly **one** inversion.
- The running time of Insertion Sort is strictly $\Theta(n + I)$.
- When an array is “nearly sorted” such that $I = O(n)$, Insertion Sort runs in strictly linear $\Theta(n)$ time.

3. Efficient Divide-and-Conquer Sorts: Merge Sort and Quick Sort

3.1 Merge Sort

Merge Sort applies the classic **Divide-and-Conquer** paradigm:

1. **Divide:** Partition array of size n into two equal sublists of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$.
2. **Conquer:** Recursively sort each sublist. Base case: a list of size ≤ 1 is already sorted.
3. **Combine:** Merge the two sorted sublists into a single sorted array.



3.1.1 Formal Merge and Merge Sort Pseudocode

```

1 FUNCTION MERGE(L, R)
2 INPUT: sorted arrays L and R
3 OUTPUT: one combined sorted array containing all values
4   result = empty array
5   i = 0; j = 0
6   WHILE i < LENGTH(L) AND j < LENGTH(R) DO
7     IF L[i] <= R[j] THEN           // <= ensures STABILITY
8       APPEND L[i] TO result; i = i + 1
9     ELSE
10      APPEND R[j] TO result; j = j + 1
11  APPEND all remaining elements of L TO result
12  APPEND all remaining elements of R TO result
13  RETURN result
14
15 FUNCTION MERGE_SORT(A)
16 INPUT: array A
17 OUTPUT: sorted array
18   IF LENGTH(A) <= 1 THEN RETURN A
19   mid = LENGTH(A) DIV 2
20   L = MERGE_SORT(A[0 : mid])
21   R = MERGE_SORT(A[mid : LENGTH(A)])
22   RETURN MERGE(L, R)
  
```

3.1.2 Mathematical Complexity Analysis

The recurrence relation for Merge Sort on an input of size n is:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Using the Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$. The watershed function is $n^{\log_b a} = n^{\log_2 2} = n^1$. Since $f(n) = \Theta(n^1)$, this falls into **Case 2**:

$$T(n) = \Theta(n \log n) \quad \text{in Best, Average, and Worst Cases!}$$

Tradeoffs:

- **Advantage:** Predictable $\Theta(n \log n)$ guarantee; completely immune to bad inputs; **stable**.
- **Disadvantage:** **Not in-place**. Merging requires $\Theta(n)$ auxiliary buffer space.

3.2 Quick Sort

Quick Sort operates by choosing an element called a **pivot**, partitioning the remaining elements into two groups—those less than or equal to the pivot, and those greater than the pivot—and then recursively sorting each group.

3.2.1 Lomuto Partition Scheme and Quicksort Pseudocode

```

1 FUNCTION QUICK_SORT(A, low, high)
2   IF low >= high THEN RETURN
3   p = PARTITION(A, low, high)
4   QUICK_SORT(A, low, p - 1)
5   QUICK_SORT(A, p + 1, high)
6
7 FUNCTION PARTITION(A, low, high)
8   pivot = A[high]           // Choose last element as pivot
9   boundary = low
10  FOR scan = low TO high - 1 DO
11    IF A[scan] <= pivot THEN
12      SWAP A[boundary], A[scan]
13      boundary = boundary + 1
14  SWAP A[boundary], A[high]   // Put pivot into correct final slot
15  RETURN boundary

```

3.2.2 Trace of Lomuto Partitioning

Trace on subarray [7, 3, 5, 2, 6, 4] with pivot = 4:

Scan Index	Scanned Value	Action Taken	Array State (<i>boundary</i> position)
Start	–	pivot = 4	[7, 3, 5, 2, 6, 4], <i>boundary</i> = 0
<i>scan</i> = 0	7	$7 > 4 \implies$ do nothing	[7, 3, 5, 2, 6, 4], <i>boundary</i> = 0
<i>scan</i> = 1	3	$3 \leq 4 \implies$ swap with A[0]	[3, 7, 5, 2, 6, 4], <i>boundary</i> = 1
<i>scan</i> = 2	5	$5 > 4 \implies$ do nothing	[3, 7, 5, 2, 6, 4], <i>boundary</i> = 1
<i>scan</i> = 3	2	$2 \leq 4 \implies$ swap with A[1]	[3, 2, 5, 7, 6, 4], <i>boundary</i> = 2
<i>scan</i> = 4	6	$6 > 4 \implies$ do nothing	[3, 2, 5, 7, 6, 4], <i>boundary</i> = 2
Finish	–	Swap A[2] with pivot A[5]	[3, 2, 4, 7, 6, 5], return <i>boundary</i> = 2

3.2.3 The Pivotal Role of Pivot Selection

- **Best Case (Balanced Split):** Pivot is always the median. $T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$.
- **Worst Case (Degenerate Split):** Naive pivot on already sorted array. Partition produces subproblems of size 0 and $n - 1$:

$$T(n) = T(n - 1) + \Theta(n) = \sum_{i=1}^n i = \Theta(n^2)$$

- **Average Case:** $T(n) = \Theta(n \log n)$ with small constant factors.
- **Solutions to Worst-Case:** (1) **Randomized Quicksort** (swap random index with ‘high’ before partitioning), (2) **Median-of-Three** (take median of $\{A[\text{low}], A[\text{mid}], A[\text{high}]\}$).
- **Stability: Unstable** due to long-distance swaps during partitioning.

3.3 Worked Examples on Divide-and-Conquer Sorting

Example 3.1 (Tracing the Two-Way Merge Step-by-Step)

Suppose the recursive calls have sorted two halves into $L = [3, 5, 7]$ and $R = [2, 4, 6]$. The ‘MERGE(L, R)’ function combines them into a single sorted array by maintaining pointers i and j :

Step	Compare $L[i]$ vs $R[j]$	Condition	Selected Item	Output Array So Far
1	$L[0] = 3$ vs $R[0] = 2$	$3 > 2$	$R[0] = 2$, advance $j \rightarrow 1$	[2]
2	$L[0] = 3$ vs $R[1] = 4$	$3 \leq 4$	$L[0] = 3$, advance $i \rightarrow 1$	[2, 3]
3	$L[1] = 5$ vs $R[1] = 4$	$5 > 4$	$R[1] = 4$, advance $j \rightarrow 2$	[2, 3, 4]
4	$L[1] = 5$ vs $R[2] = 6$	$5 \leq 6$	$L[1] = 5$, advance $i \rightarrow 2$	[2, 3, 4, 5]
5	$L[2] = 7$ vs $R[2] = 6$	$7 > 6$	$R[2] = 6$, advance $j \rightarrow 3$	[2, 3, 4, 5, 6]
6	R is exhausted	–	Append remaining $L[2] = 7$	[2, 3, 4, 5, 6, 7]

Key Observation: Exactly 5 comparisons were needed to merge $3 + 3 = 6$ elements. In general, merging lists of size n_1 and n_2 requires at most $n_1 + n_2 - 1$ comparisons, proving the merge step is strictly $\Theta(n)$.

Example 3.2 (Quicksort Degeneracy: Naive vs Median-of-Three Pivot)

Consider sorting the already sorted sequence $A = [1, 2, 3, 4, 5]$:

- **Naive Strategy (Last Element as Pivot):**

- Call 1: Pivot = 5. Partition splits into $[1, 2, 3, 4]$ (left) and $[]$ (right). Comparisons: 4.
- Call 2: Pivot = 4. Partition splits into $[1, 2, 3]$ (left) and $[]$ (right). Comparisons: 3.
- Call 3: Pivot = 3. Partition splits into $[1, 2]$ and $[]$. Comparisons: 2.
- Call 4: Pivot = 2. Partition splits into $[1]$ and $[]$. Comparisons: 1.
- **Recursion Tree:** Completely skewed chain of depth $n - 1 = 4$. Total comparisons: $4 + 3 + 2 + 1 = 10 = \frac{n(n-1)}{2} = \Theta(n^2)$.

- **Median-of-Three Strategy:** Sample $\{A[0] = 1, A[2] = 3, A[4] = 5\}$. The median is 3. Swapping 3 to the end as pivot splits the array into $[1, 2]$ and $[4, 5]$: two perfectly balanced halves! The tree height is $\lceil \log_2 5 \rceil = 3$, and total comparisons are bounded by $O(n \log n)$.

4. Non-Comparison Linear-Time Sorting: Counting Sort & Radix Sort

4.1 The $\Omega(n \log n)$ Lower Bound for Comparison Sorts

Theorem 4.1 (Information-Theoretic Lower Bound)

Any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons in the worst case to sort n distinct elements.

Proof via Decision Trees:

1. Any comparison sort can be modeled as a binary decision tree where each internal node represents a comparison $A[i] \leq A[j]$ and each leaf represents a distinct sorted permutation of the input.
2. An array of n elements has $n!$ possible permutations. The decision tree must contain at least $n!$ reachable leaves ($L \geq n!$).
3. A binary tree of height h can have at most 2^h leaves. Therefore:

$$2^h \geq L \geq n! \implies h \geq \log_2(n!)$$

4. Applying Stirling's approximation ($\ln(n!) = n \ln n - n + O(\ln n)$):

$$h \geq \log_2(n!) = \sum_{i=1}^n \log_2 i \geq \sum_{i=n/2}^n \log_2(n/2) = \frac{n}{2}(\log_2 n - 1) = \Omega(n \log n)$$

To achieve linear $O(n)$ sorting time, an algorithm must be **non-comparison-based**, leveraging integer keys as indices.

4.2 Counting Sort

Counting Sort assumes all n input keys are non-negative integers bounded by a known integer k (i.e., keys $\in \{0, 1, \dots, k\}$).

```

1 FUNCTION COUNTING_SORT(A, k)
2 INPUT: array A of n integer keys in {0, ..., k}
3 OUTPUT: stable sorted array B
4   CREATE count[0 .. k] initialized to 0
5   FOR i = 0 TO n - 1 DO // Pass 1: Frequency histogram
6     count[A[i]] = count[A[i]] + 1
7   FOR key = 1 TO k DO // Pass 2: Prefix sums (positions)
8     count[key] = count[key] + count[key - 1]
9   CREATE output array B of length n
10  FOR i = n - 1 DOWNT0 0 DO // Pass 3: Backwards placement (STABILITY)
11    key = A[i]
12    B[count[key] - 1] = A[i]
13    count[key] = count[key] - 1
14  RETURN B

```

4.2.1 Why Counting Sort Must Scan Backwards for Stability

In Pass 3, traversing i from $n-1$ down to 0 ensures that when duplicate keys are encountered, the one appearing later in the input array receives the highest available precomputed position ('count[key] - 1'), thus preserving the initial relative ordering.

4.2.2 Complexity Analysis and Constraints

- **Time Complexity:** $\Theta(n + k)$. Allocating and zeroing ‘count’ takes $\Theta(k)$; Pass 1 takes $\Theta(n)$; Pass 2 takes $\Theta(k)$; Pass 3 takes $\Theta(n)$.
- **When is it Linear?** If $k = O(n)$, then $T(n) = \Theta(n + n) = \Theta(n)$.
- **When does it fail?** If $n = 10$ and keys are in range 0 to 1,000,000,000, Counting Sort allocates an array of 1 billion elements, requiring gigabytes of RAM and billions of operations!

4.3 Radix Sort

Radix Sort resolves Counting Sort’s memory limitation by sorting multi-digit numbers digit by digit, from **Least Significant Digit (LSD)** to Most Significant Digit (MSD).

```

1 FUNCTION RADIX_SORT(A, d, b)
2 INPUT: array A of n non-negative integers with at most d digits in base b
3 OUTPUT: array A sorted in non-decreasing order
4   FOR digit_pos = 0 TO d - 1 DO
5     A = STABLE_COUNTING_SORT_BY_DIGIT(A, digit_pos, b)
6   RETURN A

```

Theorem 4.2 (Radix Sort Correctness and Complexity)

If the underlying digit-sort subroutine is **stable**, Radix Sort correctly sorts numbers with d digits in base b in $\Theta(d(n + b))$ time using $\Theta(n + b)$ auxiliary space.

Example: To sort 32-bit integers, we can choose base $b = 2^8 = 256$ (1 byte). Any 32-bit integer has $d = 4$ bytes. Radix sort requires exactly $d = 4$ stable counting passes, each with $k = 256$, sorting n records in $\Theta(4(n + 256)) = \Theta(n)$ time!

4.4 Worked Examples: Counting Sort and Radix Sort Traces

Example 4.1 (Full Step-by-Step Counting Sort Trace)

Consider sorting the array $A = [2, 5, 3, 0, 2, 3, 0, 3]$ with $n = 8$ and maximum key $k = 5$:

- Pass 1 (Frequency Counting):** Initialize 'count[0..5] = [0, 0, 0, 0, 0, 0]'. After scanning A :

count = [2, 0, 2, 3, 0, 1] (two 0s, zero 1s, two 2s, three 3s, zero 4s, one 5)

- Pass 2 (Prefix Sums):** Accumulate running totals to determine ending positions:

count[0] = 2, count[1] = 2, count[2] = 4, count[3] = 7, count[4] = 7, count[5] = 8

- Pass 3 (Backwards Placement into $B[0..7]$):**

Step	Item $A[i]$	Target Index 'count[key]-1'	Decrement Count	Array B State
$i = 7$	$A[7] = 3$	index $7 - 1 = 6$	'count[3]' becomes 6	[_, _, _, _, _, 3 , _]
$i = 6$	$A[6] = 0$	index $2 - 1 = 1$	'count[0]' becomes 1	[_, 0 , _, _, _, 3, _]
$i = 5$	$A[5] = 3$	index $6 - 1 = 5$	'count[3]' becomes 5	[_, 0, _, _, 3 , 3, _]
$i = 4$	$A[4] = 2$	index $4 - 1 = 3$	'count[2]' becomes 3	[_, 0, _, 2 , _, 3, 3]
$i = 3$	$A[3] = 0$	index $1 - 1 = 0$	'count[0]' becomes 0	[0 , 0, _, 2, _, 3, 3]
$i = 2$	$A[2] = 3$	index $5 - 1 = 4$	'count[3]' becomes 4	[0, 0, _, 2, 3 , 3, 3]
$i = 1$	$A[1] = 5$	index $8 - 1 = 7$	'count[5]' becomes 7	[0, 0, _, 2, 3, 3, 3 , 5]
$i = 0$	$A[0] = 2$	index $3 - 1 = 2$	'count[2]' becomes 2	[0, 0, 2 , 2, 3, 3, 3, 5]

Notice how the three occurrences of 3 preserve their relative arrival order in slots 4, 5, and 6, proving stability!

Example 4.2 (Full Step-by-Step Radix Sort Trace)

Sort the list of 3-digit decimal integers $A = [170, 045, 075, 090, 802, 024, 002, 066]$ ($n = 8, d = 3, b = 10$):

- Pass 1 (Sort stably by least significant digit – units place 10^0):**

Comparing digits [0, 5, 5, 0, 2, 4, 2, 6]:

Result $A^{(1)} = [170, 090, 802, 002, 024, 045, 075, 066]$

- Pass 2 (Sort stably by middle digit – tens place 10^1):**

Comparing digits [7, 9, 0, 0, 2, 4, 7, 6] on $A^{(1)}$:

Result $A^{(2)} = [802, 002, 024, 045, 066, 170, 075, 090]$ (Notice 802 precedes 002 because 802 preceded 002 in $A^{(1)}$!)

- Pass 3 (Sort stably by most significant digit – hundreds place 10^2):**

Comparing digits [8, 0, 0, 0, 0, 1, 0, 0] on $A^{(2)}$:

Final Sorted Result $A^{(3)} = [002, 024, 045, 066, 075, 090, 170, 802]$

All elements are now completely sorted in exactly 3 passes of 8 elements each!

5. Python Sorting in Practice

5.1 Python's Built-in Sorting: 'sorted()' vs. 'list.sort()'

Python incorporates **Timsort** (engineered by Tim Peters in 2002), an adaptive hybrid of Merge Sort and Insertion Sort:

- Finds naturally occurring ordered sequences (“runs”) in $O(n)$ time.
- Uses binary insertion sort on short runs and merges balanced runs with an intelligent stack mechanism.
- Guarantees $\Theta(n \log n)$ worst-case, $\Theta(n)$ best-case on presorted data, and is strictly **stable**.

```

1 # sorted() returns a NEW sorted list (works on any iterable, original unchanged)
2 original = [7, 3, 5, 2]
3 new_list = sorted(original)           # original remains [7, 3, 5, 2]
4
5 # list.sort() sorts the list IN-PLACE and returns None
6 original.sort()                       # original is now [2, 3, 5, 7]
```

5.1.1 Multi-Attribute Sorting with Keys

Python's 'key' parameter accepts a callable that extracts a comparison key from each element before sorting:

```

1 students = [
2     {"id": "S1", "name": "Lan", "gpa": 3.4},
3     {"id": "S2", "name": "Minh", "gpa": 3.8},
4     {"id": "S3", "name": "An", "gpa": 3.8},
5 ]
6
7 # Sort by GPA descending (-gpa), then by Name ascending (s["name"])
8 ranked = sorted(students, key=lambda s: (-s["gpa"], s["name"]))
9 # Result: S3 (An, 3.8), S2 (Minh, 3.8), S1 (Lan, 3.4)
```

5.2 Practical Python Sorting Case Studies

Example 5.1 (Multi-Criteria Product Catalog Sorting)

Suppose an online retail backend receives products with multiple attributes and needs to sort by **Category** ascending, then by **Rating** descending, and finally by **Price** ascending. Because Python evaluates tuples lexicographically element-by-element, a tuple-based **key** resolves all primary, secondary, and tertiary criteria in a single $O(n \log n)$ pass.

```

1 products = [
2     {"name": "Laptop Pro", "category": "Electronics", "rating": 4.8, "price": 1200},
3     {"name": "Gaming PC", "category": "Electronics", "rating": 4.9, "price": 1800},
4     {"name": "Notebook", "category": "Books", "rating": 4.8, "price": 15},
5     {"name": "Desk Lamp", "category": "Electronics", "rating": 4.8, "price": 45},
6 ]
7
8 # Sort by Category ascending, then by Rating descending, then by Price ascending
9 sorted_products = sorted(
10     products,
11     key=lambda p: (p["category"], -p["rating"], p["price"])
12 )
```

Example 5.2 (Emulating Custom Comparison Functions)

When sorting complex geometries, intervals, or custom objects where a single-value key projection is cumbersome, we can use `functools.cmp_to_key` to adapt a two-argument comparator (a, b) to modern Python's key-based interface:

```
1 from functools import cmp_to_key
2
3 # Sort intervals [start, end]: earlier start first; if tie, longer interval first
4 intervals = [(1, 4), (2, 6), (1, 8), (3, 5)]
5
6 def compare_intervals(a, b):
7     if a[0] != b[0]:
8         return -1 if a[0] < b[0] else 1
9     # Tie in start: larger end comes first (descending)
10    return -1 if a[1] > b[1] else (1 if a[1] < b[1] else 0)
11
12 sorted_intervals = sorted(intervals, key=cmp_to_key(compare_intervals))
13 # Result: [(1, 8), (1, 4), (2, 6), (3, 5)]
```

5.3 Common Sorting Mistakes and Anti-Patterns

Common Traps in Sorting

1. **Assigning the result of `'list.sort()'`:** Writing `'x = x.sort()'` sets `'x = None'` because `'sort()'` modifies in place and returns `'None'`!
2. **Assuming Quicksort is always fast:** Using naive Quicksort on sorted data produces $O(n^2)$ time and call-stack crashes (`'RecursionError'`).
3. **Using Counting Sort on floating-point or large ranges:** Counting sort requires non-negative integers; using it for floats or 10^9 ranges causes out-of-memory errors.
4. **Breaking Stability Unintentionally:** In Merge Sort or Counting Sort, replacing `'<='` with `'<'` destroys stability.

6. The Selection Problem and Order Statistics

6.1 Order Statistics and Ranks

Definition 6.1 (The Selection Problem)

Given a collection of n elements A and an integer $k \in \{1, 2, \dots, n\}$, find the element that would occupy index $k - 1$ if the array were sorted in non-decreasing order. This element is called the **k -th order statistic** (or element of rank k).

- Rank $k = 1$: Minimum value.
- Rank $k = n$: Maximum value.
- Rank $k = \lceil n/2 \rceil$: Median.

6.2 Finding Extremes in a Single Scan: $\Theta(n)$

Finding the minimum or maximum requires only $n - 1$ comparisons:

```

1 FUNCTION MINIMUM(A)
2 INPUT: non-empty array A
3 OUTPUT: smallest value in A
4     best = A[0]
5     FOR i = 1 TO LENGTH(A) - 1 DO
6         IF A[i] < best THEN best = A[i]
7     RETURN best

```

Note: Finding *both* the minimum and maximum simultaneously can be achieved in $\lceil 3n/2 \rceil - 2$ comparisons (rather than $2n - 2$) by processing elements in pairs and comparing the larger against current max and smaller against current min.

6.3 Strategies for General Rank Selection

Strategy	Time Complexity	Practical Assessment
Sort, then index	$\Theta(n \log n)$	Ideal if multiple rank queries follow or full sorted output is required.
Repeated selection	$\Theta(k \cdot n)$	Viable only for very small k ($k \leq 3$). Inefficient for medians ($k = n/2$).
Quickselect	Expected $\Theta(n)$	Optimal in-place choice for a single rank query on an in-memory array.
Heap of size k	$\Theta(n \log k)$	Optimal for finding Top- k items in a live continuous data stream.
Median of Medians	Worst-case $\Theta(n)$	Theoretical breakthrough ensuring $O(n)$ worst-case pivot selection.

6.4 Quickselect (Hoare's Selection Algorithm)

Quickselect exploits the fact that the partition subroutine of Quicksort places the chosen pivot into its final sorted position p . If p matches our target rank, we are done; otherwise, we recurse **into only one partition**, discarding the other!

```

1 FUNCTION QUICKSELECT(A, k)
2 INPUT: array A, rank k (1 <= k <= LENGTH(A))
3 OUTPUT: the kth-smallest value in A
4     target = k - 1                                // Convert 1-based rank to 0-based index
5     low = 0; high = LENGTH(A) - 1
6     WHILE low <= high DO
7         p = PARTITION(A, low, high)                // Same partition as Quicksort

```

```

8      IF p == target THEN
9          RETURN A[p]                // Target found!
10     ELSE IF target < p THEN
11         high = p - 1                // Target is in left partition
12     ELSE
13         low = p + 1                 // Target is in right partition

```

6.4.1 Trace for 5th-Smallest Value ($k = 5 \Rightarrow$ target index 4)

Trace on $A = [7, 3, 5, 2, 6, 4]$:

Active Range	Array State	Pivot Index p	Action
[0..5]	[3, 2, 4, 7, 6, 5]	$p = 2$ (value 4)	$4 < \text{target } 4 \Rightarrow \text{low} = 3$ (Search right [7, 6, 5])
[3..5]	[3, 2, 4, 5, 6, 7]	$p = 3$ (value 5)	$5 < \text{target } 4 \Rightarrow \text{low} = 4$ (Search right [6, 7])
[4..5]	[3, 2, 4, 5, 6, 7]	$p = 4$ (value 6)	$p == \text{target } 4 \Rightarrow$ Return 6!

6.4.2 Quickselect Complexity Analysis

- **Average Case:** On each step, array size halves: $n + \frac{n}{2} + \frac{n}{4} + \dots \leq 2n = \Theta(n)$.
- **Worst Case:** Naive pivot splits 0 and $n - 1$: $n + (n - 1) + \dots + 1 = \Theta(n^2)$.

6.5 The Median of Medians Algorithm (BFPRT)

Invented by Blum, Floyd, Pratt, Rivest, and Tarjan (1973), this algorithm guarantees a good pivot deterministically:

1. Divide n elements into $\lfloor n/5 \rfloor$ groups of 5 elements (and one remainder group).
2. Find the median of each group of 5 using insertion sort ($O(1)$ per group, $O(n)$ total).
3. Recursively find the median of the $\lfloor n/5 \rfloor$ medians; call this pivot x .
4. Partition array around x . At least 30% of elements are strictly smaller than x and at least 30% are strictly larger! The remaining subproblem is guaranteed to be at most $\frac{7}{10}n$.

Recurrence Relation:

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$$

Because $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, the sum of subproblem fractions is strictly less than 1. By induction/substitution, $T(n) = \Theta(n)$ **in the absolute worst case**.

6.6 Worked Examples in Selection and Top- k Extraction

Example 6.1 (Finding the Median of 7 Numbers with Quickselect)

Find the median (rank $k = 4 \Rightarrow$ target index 3) of the unsorted array $A = [9, 4, 8, 2, 7, 1, 5]$ ($n = 7$):

1. **Call 1 (Range $\text{low} = 0, \text{high} = 6$):** Pivot $A[6] = 5$.
 - Elements ≤ 5 : 4, 2, 1. Elements > 5 : 9, 8, 7.
 - After partition: Array becomes [4, 2, 1, 5, 7, 9, 8] with pivot index $p = 3$.
 - Pivot index $p = 3$ matches our target index 3 on the very first try!
 - **Result:** Median is 5. Total comparisons: 6.

Example 6.2 (Streaming Top- k Processing via Python `heapq`)

When extracting the top $k = 3$ largest transactions from a continuous streaming log of millions of records, keeping all records in memory is prohibited. Instead, maintain a min-heap of size $k = 3$. For n stream elements, memory usage is strictly $O(k)$, and total runtime is $O(n \log k)$. For $k \ll n$, this is practically linear!

```
1 import heapq
2
3 def top_k_streaming(stream, k):
4     # Initialize heap with the first k elements
5     min_heap = []
6     for value in stream:
7         if len(min_heap) < k:
8             heapq.heappush(min_heap, value)
9         elif value > min_heap[0]:
10            # If current value exceeds the smallest in top-k, replace it
11            heapq.heappushpop(min_heap, value)
12    return sorted(min_heap, reverse=True)
13
14 # Example usage on incoming scores
15 scores = [12, 85, 45, 92, 60, 99, 74, 30]
16 print(top_k_streaming(scores, 3)) # Output: [99, 92, 85]
```

7. Searching Algorithms: Linear, Binary, and Ternary Search

7.1 Linear Search

For unsorted collections, we inspect elements sequentially from left to right:

```
1 FUNCTION LINEAR_SEARCH(A, target)
2   FOR i = 0 TO LENGTH(A) - 1 DO
3     IF A[i] == target THEN RETURN i
4   RETURN NONE
```

Complexity: Best case $\Omega(1)$ (target at index 0); Worst case $O(n)$ (target absent); Average case $\frac{n+1}{2} = \Theta(n)$.

7.2 Binary Search

Prerequisite: The collection A must be sorted in non-decreasing order.

```
1 FUNCTION BINARY_SEARCH(A, target)
2 INPUT: sorted array A, search target
3 OUTPUT: index of target, or NONE
4   low = 0
5   high = LENGTH(A) - 1
6   WHILE low <= high DO
7     mid = low + (high - low) DIV 2    // Overflow-safe midpoint
8     IF A[mid] == target THEN
9       RETURN mid
10    ELSE IF A[mid] < target THEN
11      low = mid + 1
12    ELSE
13      high = mid - 1
14  RETURN NONE
```

7.2.1 Trace for Target 6 on [2, 3, 4, 5, 6, 7]

Step	low	high	mid	$A[mid]$	Action
1	0	5	2	4	$4 < 6 \implies low = mid + 1 = 3$ (discard indices 0..2)
2	3	5	4	6	$6 == 6 \implies$ Return index 4!

7.2.2 Correctness Invariant and Complexity

- **Loop Invariant:** If ‘target’ exists in A , it must lie within the index interval $[low, high]$.
- **Termination:** Each step strictly shrinks the search interval by half: $W_{k+1} \leq \lfloor W_k/2 \rfloor$. The loop must terminate in $\leq \lfloor \log_2 n \rfloor + 1$ iterations.
- **Time Complexity:** Worst-case $\Theta(\log n)$; Best-case $\Theta(1)$; Auxiliary space $\Theta(1)$ (iterative).

7.3 Binary Search Variants: First and Last Occurrences

When duplicates exist, standard binary search returns an arbitrary match. To locate the **first occurrence** (useful for range queries and lower bounds):

```
1 FUNCTION FIRST_OCCURRENCE(A, target)
2   low = 0; high = LENGTH(A) - 1; answer = NONE
3   WHILE low <= high DO
4     mid = low + (high - low) DIV 2
5     IF A[mid] >= target THEN
6       IF A[mid] == target THEN answer = mid
```

```

7         high = mid - 1                // Keep searching left for earlier occurrence
8     ELSE
9         low = mid + 1
10    RETURN answer

```

7.4 Python's 'bisect' Library

```

1  from bisect import bisect_left, bisect_right, insort
2
3  values = [2, 4, 4, 4, 7, 9]
4
5  # bisect_left locates the first index where target could be inserted
6  i = bisect_left(values, 4)          # i = 1 (index of first 4)
7
8  # bisect_right locates the index after the last occurrence
9  j = bisect_right(values, 4)         # j = 4 (index after last 4)
10
11 # Count occurrences in O(log n) time:
12 count_4 = j - i                     # 4 - 1 = 3 occurrences
13
14 # insort inserts into sorted list maintaining sorted order (O(n) shift)
15 insort(values, 5)                   # values becomes [2, 4, 4, 4, 5, 7, 9]

```

7.5 Ternary Search on Unimodal Functions

A function $f(x)$ is **unimodal** on $[L, R]$ if it is strictly increasing to a unique maximum, then strictly decreasing.

- Choose two internal trisection points: $m_1 = L + (R - L)/3$ and $m_2 = R - (R - L)/3$.
- If $f(m_1) < f(m_2)$, the peak cannot be in $[L, m_1]$; update $L = m_1$.
- Otherwise, the peak cannot be in $[m_2, R]$; update $R = m_2$.
- **Recurrence:** $T(n) = T(2n/3) + O(1) \implies \Theta(\log_{1.5} n)$.

7.6 Comprehensive Data Structure Selection Matrix for Lookups

Data Structure	Search Time	Insert/Delete Time	Optimal System Context
Unsorted List	$\Theta(n)$	$O(1)$ append	Rare searches, frequent fast appends, small data.
Sorted Array	$\Theta(\log n)$	$\Theta(n)$	Static data, many read lookups, range queries.
Hash Set / Dict	Expected $O(1)$	Expected $O(1)$	Exact-key lookups, fast inserts, no range ordering.
Balanced BST	$O(\log n)$	$O(\log n)$	Dynamic collections requiring both fast updates and range queries.

7.7 Worked Examples in Searching Algorithms

Example 7.1 (Efficient Range Queries via bisect)

Given a sorted catalog of product prices, find all items priced within the interval $[\$50, \$150]$. Finding the target slice indices takes only $2 \times O(\log n) = O(\log n)$ time, regardless of how large the catalog is:

```

1 from bisect import bisect_left, bisect_right
2
3 prices = [12, 25, 40, 55, 75, 99, 120, 150, 210, 350]
4
5 # Find index of first item >= 50
6 left_idx = bisect_left(prices, 50)      # returns index 3 (value 55)
7
8 # Find index of first item > 150
9 right_idx = bisect_right(prices, 150)   # returns index 8 (value 210)
10
11 # Slice in O(k) time where k is number of matching items:
12 matched_prices = prices[left_idx:right_idx]
13 # Result: [55, 75, 99, 120, 150] (count: 8 - 3 = 5 items)

```

Example 7.2 (Discrete Peak Finding via Ternary Search)

Find the maximum element in the unimodal array $A = [1, 4, 9, 16, 25, 36, 49, 40, 28, 11]$ ($n = 10$):

- Step 1: $low = 0, high = 9$. $m_1 = 0 + 9/3 = 3$ ($A[3] = 16$); $m_2 = 9 - 9/3 = 6$ ($A[6] = 49$). Since $A[m_1] = 16 < A[m_2] = 49$, the peak cannot lie in $[0..3]$. Update $low = m_1 + 1 = 4$.
- Step 2: $low = 4, high = 9$. Range is $[25, 36, 49, 40, 28, 11]$. Trisecting narrows search to the neighborhood around index 6.
- Step 3: When interval length ≤ 3 , a small linear scan identifies $A[6] = 49$ as the maximum in ≈ 5 evaluations instead of 10.

Example 7.3 (Binary Search on Answer: Integer Square Root)

Compute $\lfloor \sqrt{x} \rfloor$ for $x = 2,000,000,000$ without using floating-point math. By binary-searching over the search space $[1, \lfloor x/2 \rfloor]$, we determine the answer in exactly $\lceil \log_2(10^9) \rceil \approx 30$ iterations.

```

1 def integer_sqrt(x):
2     if x < 2:
3         return x
4     low, high = 1, x // 2
5     ans = 1
6     while low <= high:
7         mid = low + (high - low) // 2
8         if mid * mid <= x:
9             ans = mid          # mid is a candidate
10            low = mid + 1       # try to find a larger answer
11        else:
12            high = mid - 1      # mid * mid > x, search left
13    return ans
14
15 print(integer_sqrt(2000000000)) # Output: 44721 (since 44721^2 <= 2*10^9 < 44722^2)

```

8. Synthesis Problems and Worked Solutions

8.1 Problem 1: Transaction Record Multi-Key Sorting

Problem: Given a list of transactions, sort them in chronological order by ‘date’ (ascending); for transactions on the same date, sort by ‘amount’ descending; and for identical dates and amounts, preserve their original relative arrival order.

```
1 transactions = [
2     {"id": "T1", "date": "2026-08-10", "amount": 250},
3     {"id": "T2", "date": "2026-08-09", "amount": 400},
4     {"id": "T3", "date": "2026-08-10", "amount": 300},
5     {"id": "T4", "date": "2026-08-10", "amount": 250},
6 ]
```

Solution to Problem 1

Because Python’s Timsort is guaranteed to be **stable**, we can sort in a single call by constructing a tuple key: `key=lambda t: (t["date"], -t["amount"])`. Stability guarantees that for transactions having the exact same date and amount, their original relative arrival order is strictly preserved. Time complexity is $\Theta(n \log n)$.

```
1 result = sorted(transactions, key=lambda t: (t["date"], -t["amount"]))
2 # Output:
3 # 1. T2: 2026-08-09, amount 400
4 # 2. T3: 2026-08-10, amount 300
5 # 3. T1: 2026-08-10, amount 250 (preceded T4 in original list)
6 # 4. T4: 2026-08-10, amount 250
```

8.2 Problem 2: Choosing Top- k Strategy

Problem: Compare three strategies to find the k largest values in an unsorted array of size $n = 10,000,000$ when $k = 100$: (A) Full Sort, (B) Quickselect, (C) Min-Heap of size k .

Solution to Problem 2

- **Strategy A (Full Sort):** $\Theta(n \log n) \approx 10^7 \times 24 \approx 2.4 \times 10^8$ operations. Extremely wasteful since we only need the top 100 values.
- **Strategy B (Quickselect):** Expected $\Theta(n)$ time $\approx 10^7$ operations. In-place, but modifies the input array.
- **Strategy C (Min-Heap of size k):** Maintain a min-heap of size $k = 100$. For each incoming element, compare with the heap top ($O(1)$); if larger, replace top in $O(\log k)$ time. Total operations: $n \log_2 k \approx 10^7 \times 7 = 7 \times 10^7$.
- **Recommendation:** If data fits in RAM, Quickselect is fastest ($\Theta(n)$). If data is a **stream** or RAM is limited, a **Min-Heap of size k** is superior because it requires only $O(k) = O(100)$ auxiliary space!

8.3 Problem 3: Two-Sum on a Sorted Array

Problem: Given a 1-indexed array of integers A that is already sorted in non-decreasing order, find two distinct indices $i < j$ such that $A[i] + A[j] = \text{target}$. Design an algorithm using $O(1)$ extra memory and $O(n)$ time.

Solution to Problem 3

Because A is sorted, we employ the **Two-Pointer Technique**. At each iteration, either `left` advances or `right` retreats. The distance $right - left$ decreases by at least 1 per step, guaranteeing at most n comparisons. Time complexity is $\Theta(n)$, and auxiliary space is $\Theta(1)$ (superior to an unsorted Hash Table which requires $O(n)$ memory).

```

1 def two_sum_sorted(numbers, target):
2     left = 0
3     right = len(numbers) - 1
4     while left < right:
5         current_sum = numbers[left] + numbers[right]
6         if current_sum == target:
7             return (left + 1, right + 1)    # 1-indexed result
8         elif current_sum < target:
9             left += 1                        # Need a larger sum
10        else:
11            right -= 1                       # Need a smaller sum
12    return None

```

8.4 Problem 4: Searching in a Rotated Sorted Array

Problem: An array sorted in ascending order is rotated at an unknown pivot index (e.g., [0, 1, 2, 4, 5, 6, 7] becomes [4, 5, 6, 7, 0, 1, 2]). Given a target value, find its index in $O(\log n)$ time.

Solution to Problem 4

Even though the array is rotated, **at least one half is always strictly sorted!** In every iteration, the search interval is cut in half by identifying the sorted subarray and checking whether the target falls within its range. Running time is strictly $O(\log n)$ and auxiliary space is $O(1)$.

```

1 def search_rotated(A, target):
2     low = 0
3     high = len(A) - 1
4     while low <= high:
5         mid = low + (high - low) // 2
6         if A[mid] == target:
7             return mid
8
9         # Check if left half A[low..mid] is sorted
10        if A[low] <= A[mid]:
11            if A[low] <= target < A[mid]:
12                high = mid - 1    # Target lies in sorted left half
13            else:
14                low = mid + 1     # Target lies in right half
15        # Otherwise, right half A[mid..high] must be sorted
16        else:
17            if A[mid] < target <= A[high]:
18                low = mid + 1     # Target lies in sorted right half
19            else:
20                high = mid - 1    # Target lies in left half
21    return -1

```